

GOOGLE CLOUD PROFESSIONAL CLOUD ARCHITECT

Section 3.2 — Deploying and Implementing Google Kubernetes Engine Resources

GCP Associate Cloud Engineer

Study Notes Study Notes April 2026

Exam Relevance

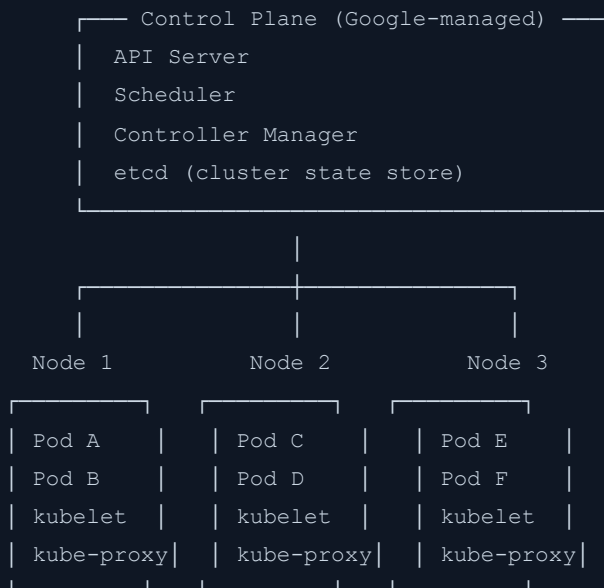
This topic is part of **Section 3: Deploying and implementing a cloud solution (~25 % of the exam)**. You must know how to install and configure kubectl, deploy GKE clusters with different configurations, and deploy containerized applications.

1. Kubernetes Fundamentals

Core Concepts

Concept	Description
Cluster	A set of nodes (VMs) running containerized applications managed by Kubernetes
Control Plane	Manages the cluster (API server, scheduler, controller manager, etcd)
Node	A worker VM that runs pods
Pod	Smallest deployable unit — one or more containers sharing network/storage
Deployment	Manages a set of identical pods with declarative updates
Service	Stable network endpoint to access a group of pods
Namespace	Virtual cluster within a physical cluster for resource isolation
ConfigMap	Store non-sensitive configuration data
Secret	Store sensitive data (passwords, tokens, keys)
Ingress	HTTP/S routing rules to services
PersistentVolume	Storage resource provisioned for pods

Architecture



2. Installing and Configuring kubectl

What Is kubectl?

- **Command-line tool** for interacting with Kubernetes clusters
- Sends commands to the cluster's API server
- Can manage pods, services, deployments, and all Kubernetes resources

Installation

```

# Install kubectl via gcloud
gcloud components install kubectl

# Verify installation
kubectl version --client

# Alternative: install via package manager
# macOS
brew install kubectl

# Linux
sudo apt-get install -y kubectl
  
```

Configuring kubectl to Access a GKE Cluster

```
# Get credentials for a GKE cluster (updates ~/.kube/config)
gcloud container clusters get-credentials CLUSTER_NAME \
  --zone=us-central1-a \
  --project=PROJECT_ID

# For regional clusters
gcloud container clusters get-credentials CLUSTER_NAME \
  --region=us-central1 \
  --project=PROJECT_ID

# Verify connection
kubectl cluster-info
kubectl get nodes
```

kubeconfig File

- Located at `~/.kube/config`
- Contains cluster connection info, credentials, and contexts
- **Context** = cluster + user + namespace combination

```
# View current context
kubectl config current-context

# List all contexts
kubectl config get-contexts

# Switch context
kubectl config use-context CONTEXT_NAME

# Set default namespace for current context
kubectl config set-context --current --namespace=my-namespace
```

3. Deploying a GKE Cluster

GKE Cluster Modes

Feature	Standard	Autopilot
Node management	You manage nodes and node pools	Google manages nodes
Pricing	Pay for node VMs	Pay per pod resources
Node configuration	Full control (machine types, GPUs, etc.)	Google-optimized
Security hardening	Manual (you apply best practices)	Automatic (enforced)
Scaling	Cluster autoscaler + manual	Fully automatic
Resource efficiency	May have unused node capacity	Optimized bin-packing
Operational overhead	Higher	Lower
Best for	Custom requirements, GPU workloads, full control	Most workloads, reduced ops

Creating a Standard Cluster

```
# Basic standard cluster
gcloud container clusters create my-cluster \
  --zone=us-central1-a \
  --num-nodes=3 \
  --machine-type=e2-standard-4

# Production-grade standard cluster
gcloud container clusters create prod-cluster \
  --zone=us-central1-a \
  --num-nodes=3 \
  --machine-type=n2-standard-4 \
  --enable-autoscaling \
  --min-nodes=2 \
  --max-nodes=10 \
  --enable-autorepair \
  --enable-autoupgrade \
  --enable-ip-alias \
  --enable-network-policy \
  --disk-type=pd-ssd \
  --disk-size=100GB \
  --release-channel=regular
```

Creating an Autopilot Cluster

```
# Autopilot cluster (simplified — Google manages nodes)
gcloud container clusters create-auto my-autopilot-cluster \
  --region=us-central1 \
  --project=PROJECT_ID

# Autopilot with private cluster
gcloud container clusters create-auto my-private-autopilot \
  --region=us-central1 \
  --enable-private-nodes \
  --enable-master-authorized-networks \
  --master-authorized-networks=10.0.0.0/8
```

Regional Clusters

Regional clusters provide **high availability** by distributing the control plane and nodes across multiple zones.

```
# Create a regional cluster (3 control plane replicas across zones)
gcloud container clusters create my-regional-cluster \
  --region=us-central1 \
  --num-nodes=2 \
  --machine-type=e2-standard-4

# This creates:
# - 3 control plane replicas (one per zone)
# - 2 nodes per zone = 6 total nodes (across 3 zones)
```

Key differences: - **Zonal cluster:** Single control plane in one zone (SPOF for control plane) - **Regional cluster:** 3 control plane replicas across 3 zones (HA for control plane) - Regional clusters cost more (3x control plane, nodes across zones) - Regional clusters survive zone failures

Private Clusters

Private clusters restrict access to the cluster's master endpoint and/or nodes.

```
# Create a private cluster
gcloud container clusters create my-private-cluster \
  --zone=us-central1-a \
  --enable-private-nodes \
  --master-ipv4-cidr=172.16.0.0/28 \
  --enable-ip-alias \
  --enable-master-authorized-networks \
  --master-authorized-networks=10.0.0.0/8,203.0.113.0/24

# Private cluster properties:
# --enable-private-nodes: Nodes get only internal IPs (no external)
# --master-ipv4-cidr: Internal IP range for the control plane
# --enable-master-authorized-networks: Restrict who can reach the API server
# --master-authorized-networks: CIDR ranges allowed to access the master
```

Private cluster features: - Nodes have **no external IP addresses** (use Cloud NAT for outbound internet) - Control plane has a **private endpoint** (internal IP) - Can optionally enable a **public endpoint** with authorized networks - Provides stronger network security

GKE Enterprise (formerly Anthos)

GKE Enterprise extends GKE to **multi-cloud and on-premises** environments:

- **GKE on AWS/Azure** — Run GKE clusters on other cloud providers
- **GKE on bare metal** — Run GKE on your own hardware
- **Config Sync** — GitOps-based configuration management

- **Policy Controller** — Enforce policies across clusters
- **Service Mesh** — Anthos Service Mesh (based on Istio)
- **Multi-cluster management** — Manage fleets of clusters centrally

4. Release Channels

GKE offers release channels to control how clusters are upgraded:

Channel	Description	Update Frequency
Rapid	Latest Kubernetes version, earliest access to features	Most frequent
Regular (default)	Balanced — access to features after stability in Rapid	Moderate
Stable	Most tested, production-ready	Least frequent
Extended	Longer support for a specific version	Patch-only

```
# Create cluster with a specific release channel
gcloud container clusters create my-cluster \
  --zone=us-central1-a \
  --release-channel=stable

# Change release channel
gcloud container clusters update my-cluster \
  --zone=us-central1-a \
  --release-channel=regular
```


5. Deploying a Containerized Application to GKE

Step 1: Build and Push a Container Image

```
# Build a Docker image
docker build -t my-app:v1 .

# Tag the image for Artifact Registry
docker tag my-app:v1 us-centrall-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1

# Push to Artifact Registry
docker push us-centrall-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1

# Alternative: Build with Cloud Build (no local Docker needed)
gcloud builds submit --tag us-centrall-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1
```

Step 2: Create a Deployment

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  labels:
    app: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app
        image: us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1
        ports:
        - containerPort: 8080
        resources:
          requests:
            cpu: "250m"
            memory: "256Mi"
          limits:
            cpu: "500m"
            memory: "512Mi"
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
            initialDelaySeconds: 30
            periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
            initialDelaySeconds: 5
            periodSeconds: 5
```

```
# Apply the deployment
kubectl apply -f deployment.yaml

# Or create directly with kubectl
kubectl create deployment my-app \
  --image=us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1 \
  --replicas=3

# Check deployment status
kubectl get deployments
kubectl rollout status deployment/my-app
```

Step 3: Expose the Application with a Service

```
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

```
# Apply the service
kubectl apply -f service.yaml

# Or expose directly
kubectl expose deployment my-app \
  --type=LoadBalancer \
  --port=80 \
  --target-port=8080

# Get external IP
kubectl get service my-app-service
```

Service Types

Type	Description	Use Case
ClusterIP (default)	Internal IP only	Internal services
NodePort	Expose on each node’s IP at a static port	Development, testing
LoadBalancer	External load balancer (GCP Network LB)	External-facing services
ExternalName	Maps to a DNS name	External service aliasing

6. Common kubectl Commands for the Exam

```
# — Cluster Information —
kubectl cluster-info
kubectl get nodes
kubectl get namespaces

# — Deployments —
kubectl get deployments
kubectl describe deployment my-app
kubectl scale deployment my-app --replicas=5
kubectl rollout history deployment/my-app
kubectl rollout undo deployment/my-app
kubectl set image deployment/my-app my-app=my-app:v2

# — Pods —
kubectl get pods
kubectl get pods -o wide           # Show node and IP
kubectl describe pod POD_NAME
kubectl logs POD_NAME
kubectl logs POD_NAME -c CONTAINER_NAME  # Multi-container pod
kubectl exec -it POD_NAME -- /bin/bash   # Shell into a pod
kubectl delete pod POD_NAME

# — Services —
kubectl get services
kubectl describe service my-service
kubectl delete service my-service

# — ConfigMaps and Secrets —
kubectl create configmap my-config --from-literal=key1=value1
kubectl create secret generic my-secret --from-literal=password=s3cr3t
kubectl get configmaps
kubectl get secrets

# — Namespaces —
kubectl create namespace my-namespace
kubectl get pods --namespace=my-namespace
kubectl get pods --all-namespaces

# — Apply and Delete —
kubectl apply -f manifest.yaml
kubectl delete -f manifest.yaml
```

7. Workload Identity

What Is Workload Identity?

- The recommended way for GKE workloads to access Google Cloud APIs
- Maps Kubernetes service accounts to Google Cloud service accounts
- Eliminates the need to manage service account keys

How It Works

```
Pod → K8s Service Account → Workload Identity → GCP Service Account → GCP APIs
```

```
# Enable Workload Identity on the cluster
gcloud container clusters update my-cluster \
  --zone=us-central1-a \
  --workload-pool=PROJECT_ID.svc.id.goog

# Create a GCP service account
gcloud iam service-accounts create my-gcp-sa

# Grant the GCP SA necessary permissions
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="serviceAccount:my-gcp-sa@PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"

# Allow K8s SA to impersonate GCP SA
gcloud iam service-accounts add-iam-policy-binding \
  my-gcp-sa@PROJECT_ID.iam.gserviceaccount.com \
  --role="roles/iam.workloadIdentityUser" \
  --member="serviceAccount:PROJECT_ID.svc.id.goog[NAMESPACE/KSA_NAME]"

# Annotate the K8s service account
kubectl annotate serviceaccount KSA_NAME \
  --namespace=NAMESPACE \
  iam.gke.io/gcp-service-account=my-gcp-sa@PROJECT_ID.iam.gserviceaccount.com
```

9. Node Pools

- A node pool is a group of nodes within a cluster that share the same configuration
- Use multiple node pools for mixed workloads (e.g., CPU-intensive vs. GPU vs. spot nodes)

```
# Add a node pool to existing cluster
gcloud container node-pools create my-pool --cluster=MY_CLUSTER --zone=ZONE --machine-type=...
# GPU node pool
gcloud container node-pools create gpu-pool --cluster=MY_CLUSTER --zone=ZONE --machine-type=...
# Spot VM node pool
gcloud container node-pools create spot-pool --cluster=MY_CLUSTER --zone=ZONE --spot --num-nodes=...
# List/describe/delete node pools
gcloud container node-pools list --cluster=MY_CLUSTER --zone=ZONE
gcloud container node-pools describe MY_POOL --cluster=MY_CLUSTER --zone=ZONE
gcloud container node-pools delete MY_POOL --cluster=MY_CLUSTER --zone=ZONE
```

- Taints: prevent pods from scheduling on a node unless they tolerate the taint; useful for dedicated hardware

10. Persistent Storage in GKE

PersistentVolume and PersistentVolumeClaim

- PV: a piece of storage in the cluster (e.g., a GCE Persistent Disk)
- PVC: a request for storage by a pod
- StorageClass: defines how to dynamically provision PVs

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: pd.csi.storage.gke.io
parameters:
  type: pd-ssd
reclaimPolicy: Retain
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: fast-ssd
  resources:
    requests:
      storage: 10Gi
```

StatefulSets with volumeClaimTemplates

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: my-db
spec:
  serviceName: my-db
  replicas: 3
  selector:
    matchLabels:
      app: my-db
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: [ReadWriteOnce]
      storageClassName: fast-ssd
      resources:
        requests:
          storage: 10Gi
```

- **Exam tip:** ReadWriteOnce (single node), ReadOnlyMany (multiple nodes read), ReadWriteMany (multiple nodes read/write — requires Filestore)

11. RBAC in GKE

- Kubernetes RBAC controls access within the cluster (separate from GCP IAM)
- Role: namespaced permissions; ClusterRole: cluster-wide permissions
- RoleBinding: binds a role to a subject within a namespace; ClusterRoleBinding: cluster-wide


```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: production
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: production
subjects:
- kind: User
  name: alice@example.com
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io

```

- GCP IAM → GKE cluster access (who can run kubectl); Kubernetes RBAC → what they can do inside the cluster
- **Exam tip:** `roles/container.admin` gives cluster admin; `roles/container.developer` gives read/write to most resources but no cluster management

Exam Practice Questions

1. **You need to create a highly available GKE cluster that survives zone failures. How should you configure it?**
 - Answer: Create a **regional cluster** (`--region=us-central1`). This distributes the control plane across 3 zones and nodes across multiple zones.
2. **Your organization requires that GKE nodes have no external IP addresses. What type of cluster should you create?**
 - Answer: A **private cluster** with `--enable-private-nodes` . Use **Cloud NAT** for outbound internet access if needed.
3. **A developer cannot run kubectl commands against the cluster. They get authentication errors. What's likely missing?**

- Answer: They need to run `gcloud container clusters get-credentials CLUSTER_NAME` to configure kubectl, and they need appropriate IAM roles (e.g., `roles/container.developer`).
4. You want to minimize operational overhead for a GKE cluster running standard web applications. Which cluster mode should you use?
- Answer: **Autopilot** — Google manages nodes, security hardening, and scaling. You only pay for pod resources.
5. How should a GKE pod access Cloud Storage without using service account keys?
- Answer: Use **Workload Identity** — map the Kubernetes service account to a Google Cloud service account with Storage permissions.
6. You need to expose a web application running in GKE to the internet. What Kubernetes resource should you create?
- Answer: Create a **Service** of type `LoadBalancer` (for L4) or an **Ingress** resource (for L7 with HTTP/S routing).
-

Glossary

API Server — The front-end component of the Kubernetes control plane that exposes the Kubernetes API; all kubectl commands and internal components communicate through it.

Artifact Registry — GCP's fully managed service for storing, managing, and securing container images and other build artifacts. Replaces Container Registry as the recommended image repository.

Autopilot (GKE) — A GKE cluster mode in which Google fully manages nodes, node pools, security hardening, and scaling. Users pay per pod resource consumption rather than per node VM.

Autorepair — A GKE feature (`--enable-autorepair`) that automatically repairs unhealthy nodes by draining and recreating them.

Autoupgrade — A GKE feature (`--enable-autoupgrade`) that automatically upgrades node pools to the current cluster version during maintenance windows.

Avro — A row-based, binary data serialization format commonly used with Dataflow and BigQuery data loads.

BGP (Border Gateway Protocol) — A routing protocol used by Cloud Router to dynamically exchange routes between a GCP VPC and on-premises or other cloud networks.

bin-packing — An optimization technique in which workloads are scheduled onto nodes to maximize resource utilization and minimize idle capacity. GKE Autopilot applies this automatically.

cbt — The command-line tool for interacting with Cloud Bigtable; used to create tables, column families, and read/write data rows.

Cloud Build — GCP's fully managed CI/CD service that builds container images from source code. Used with `gcloud builds submit` to produce images for GKE and Cloud Run without a local Docker daemon.

Cloud NAT — A GCP managed Network Address Translation service that provides outbound internet connectivity for resources (such as private GKE nodes) that have no external IP addresses.

ClusterIP — The default Kubernetes Service type that assigns an internal IP address accessible only within the cluster; used for internal service-to-service communication.

ClusterRole — A Kubernetes RBAC resource that defines a set of permissions applicable cluster-wide, across all namespaces.

ClusterRoleBinding — A Kubernetes RBAC resource that binds a ClusterRole to a subject (user, group, or service account) at the cluster scope.

Config Sync — A GKE Enterprise feature that enables GitOps-based configuration management, synchronizing Kubernetes configuration from a Git repository to one or more clusters.

ConfigMap — A Kubernetes resource used to store non-sensitive configuration data as key-value pairs, injected into pods as environment variables or mounted files.

Container — A lightweight, portable, self-sufficient unit of software that packages an application and its dependencies. GKE runs containers inside pods.

Container Image — A read-only template used to create containers, stored in a registry such as Artifact Registry. Specified in Kubernetes Deployment manifests.

Context (kubeconfig) — A named combination of cluster, user credentials, and default namespace stored in `~/.kube/config` that `kubectl` uses to target a specific cluster.

Control Plane — The set of Kubernetes components (API Server, Scheduler, Controller Manager, etcd) that manage cluster state. In GKE, the control plane is fully Google-managed.

Controller Manager — The Kubernetes control plane component that runs controllers (e.g., Deployment controller, ReplicaSet controller) that reconcile actual cluster state with desired state.

Deployment (Kubernetes) — A Kubernetes resource that manages a set of identical, stateless pod replicas with declarative updates, rollbacks, and scaling.

Docker — A platform for building, shipping, and running containers. Used locally to build and tag images before pushing to Artifact Registry for deployment on GKE.

etcd — A distributed key-value store used by the Kubernetes control plane as its backing data store for all cluster state.

Eventarc — GCP's unified event routing service that routes events from 100+ Google Cloud sources (Audit Logs, Pub/Sub, Cloud Storage, etc.) to Cloud Run, Cloud Functions, GKE, and Workflows.

ExternalName (Service type) — A Kubernetes Service type that maps a service to an external DNS name, acting as a CNAME alias within the cluster.

GCP (Google Cloud Platform) — Google's suite of cloud computing services, including compute, storage, networking, databases, analytics, and machine learning.

gcloud — The primary command-line tool for interacting with GCP services, part of the Google Cloud SDK.

GKE (Google Kubernetes Engine) — GCP's fully managed Kubernetes service that automates cluster provisioning, scaling, upgrading, and security hardening.

GKE Enterprise (formerly Anthos) — An extended GKE offering that supports multi-cloud and on-premises Kubernetes cluster management, Config Sync, Policy Controller, and Anthos Service Mesh.

GPU (Graphics Processing Unit) — A specialized processor for parallel computation and machine learning. GKE node pools can be configured with GPU accelerators for ML workloads.

HA (High Availability) — Design approaches that minimize downtime by eliminating single points of failure. Regional GKE clusters provide HA by distributing the control plane across multiple zones.

HBase — An open-source, wide-column NoSQL database modeled after Google Bigtable. Cloud Bigtable provides an HBase-compatible API.

IAM (Identity and Access Management) — GCP's system for controlling which principals can perform which actions on which resources. Controls cluster-level access (who can run kubectl) in GKE.

Ingress (Kubernetes) — A Kubernetes resource that defines HTTP/S routing rules to route external traffic to internal services, typically using a GKE-managed Google Cloud load balancer.

IP Alias — A GKE networking mode (`--enable-ip-alias`) that assigns pods their own IP addresses from a secondary VPC subnet range, enabling VPC-native routing without NAT.

Istio — An open-source service mesh that provides traffic management, observability, and security for microservices. The basis for Anthos Service Mesh in GKE Enterprise.

kubectl — The command-line tool for interacting with Kubernetes clusters. It communicates with the cluster's API Server to manage pods, deployments, services, and other resources.

kubeconfig — A YAML file (typically at `~/.kube/config`) that stores cluster connection details, user credentials, and contexts for kubectl.

kubelet — An agent running on each Kubernetes node that ensures containers described in pod specifications are running and healthy.

kube-proxy — A network proxy running on each Kubernetes node that maintains network rules to allow communication to pods from inside and outside the cluster.

Label (Kubernetes) — A key-value pair attached to Kubernetes objects (pods, nodes, services) used for selection, grouping, and filtering by controllers and services.

Liveness Probe — A Kubernetes container health check that determines whether a container is running correctly; if it fails, the kubelet restarts the container.

LoadBalancer (Service type) — A Kubernetes Service type that provisions a GCP external Network Load Balancer, assigning an external IP through which the service is accessible from the internet.

Master Authorized Networks — A GKE private cluster feature that restricts which CIDR ranges can access the cluster's API Server endpoint.

Namespace — A Kubernetes mechanism for isolating groups of resources within a single cluster, providing scope for names and enabling multi-team or multi-environment separation.

Network Policy — A Kubernetes resource that controls which pods can communicate with each other and with external endpoints, enforced at the network level.

Node — A worker machine (VM) in a Kubernetes cluster that runs pods. In GKE Standard, nodes are Compute Engine VM instances managed by the user.

Node Pool — A group of nodes within a GKE cluster that share the same machine type, image, and configuration. A cluster can have multiple node pools for different workload types.

NodePort (Service type) — A Kubernetes Service type that exposes the service on each node's IP at a static port, typically used for development and testing.

NVIDIA Tesla T4 — An NVIDIA GPU model supported on GKE node pools for machine learning inference and training workloads.

PersistentVolume (PV) — A Kubernetes cluster-level storage resource (e.g., a GCE Persistent Disk) provisioned for use by pods, independently of individual pod lifecycles.

PersistentVolumeClaim (PVC) — A Kubernetes request for storage by a pod. The cluster binds the PVC to a matching PersistentVolume or dynamically provisions one via a StorageClass.

Pod — The smallest deployable unit in Kubernetes: one or more containers sharing the same network namespace, IP address, and storage volumes.

Policy Controller — A GKE Enterprise component that enforces organizational policies across fleets of Kubernetes clusters using the Open Policy Agent (OPA) Gatekeeper framework.

Private Cluster — A GKE cluster configuration in which nodes are assigned only internal IP addresses (no external IPs) and the control plane endpoint can be made private, improving

security.

Pub/Sub — GCP's fully managed, scalable messaging service used for event-driven architectures. Cloud Functions and Cloud Run can be triggered by Pub/Sub messages.

RBAC (Role-Based Access Control) — A Kubernetes authorization mechanism that controls what operations users and service accounts can perform on cluster resources, using Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings.

ReadinessProbe — A Kubernetes container health check that determines whether a container is ready to receive traffic. Traffic is not sent to a pod until its readiness probe passes.

ReadOnlyMany (ROX) — A Kubernetes PersistentVolume access mode allowing the volume to be mounted read-only by multiple nodes simultaneously.

ReadWriteMany (RWX) — A Kubernetes PersistentVolume access mode allowing the volume to be mounted read-write by multiple nodes. In GCP, requires Cloud Filestore as the backing storage.

ReadWriteOnce (RWO) — A Kubernetes PersistentVolume access mode allowing the volume to be mounted read-write by a single node at a time. Used with GCE Persistent Disks.

Regional Cluster — A GKE cluster configuration that replicates the control plane across three zones within a region, providing high availability. Node pools are also distributed across zones.

Release Channel — A GKE feature that manages automatic Kubernetes version upgrades. Channels include Rapid, Regular, Stable, and Extended, offering different trade-offs between feature access and stability.

Replica — One instance of a pod managed by a Deployment or StatefulSet. A Deployment maintains a specified number of running replicas.

Role (Kubernetes RBAC) — A namespaced Kubernetes RBAC resource that defines a set of permissions (verbs on resources) within a specific namespace.

RoleBinding — A Kubernetes RBAC resource that binds a Role to a subject (user, group, or service account) within a specific namespace.

Scheduler (Kubernetes) — The Kubernetes control plane component that assigns newly created pods to nodes based on resource requirements, constraints, and policies.

Secret — A Kubernetes resource for storing sensitive data (passwords, tokens, keys) in a base64-encoded form, injected into pods as environment variables or mounted files.

Service (Kubernetes) — A Kubernetes resource that provides a stable network endpoint (IP address and DNS name) to access a dynamic set of pods, enabling load balancing across pod replicas.

Service Account (Kubernetes) — An identity used by pods to authenticate API calls to the Kubernetes API Server or, via Workload Identity, to GCP APIs.

Service Mesh — An infrastructure layer that manages service-to-service communication (traffic management, observability, security). GKE Enterprise provides Anthos Service Mesh based on Istio.

SPOF (Single Point of Failure) — A component whose failure would bring down an entire system. Zonal GKE clusters have an SPOF in the control plane; regional clusters eliminate this.

Spot VM — A Compute Engine pricing model offering deep discounts for fault-tolerant workloads, with no guaranteed runtime. GKE supports Spot VM node pools (`--spot`).

Standard (GKE) — The GKE cluster mode in which the user manages node configuration, node pools, and scaling. Provides full control over infrastructure at the cost of higher operational overhead.

StatefulSet — A Kubernetes workload resource for managing stateful applications (e.g., databases). Pods have stable network identities and use per-pod PersistentVolumeClaims via `volumeClaimTemplates` .

StorageClass — A Kubernetes resource that defines how PersistentVolumes are dynamically provisioned. In GKE, the `pd.csi.storage.gke.io` provisioner creates GCE Persistent Disks.

Taint — A Kubernetes node attribute that repels pods from being scheduled on it unless a pod explicitly declares a toleration for that taint. Used to dedicate nodes for specific workloads.

VPC (Virtual Private Cloud) — GCP's global, software-defined private network. GKE clusters are deployed within a VPC, using subnets for node and pod IP address allocation.

VPC-native cluster — A GKE cluster using IP aliasing (`--enable-ip-alias`) where pods receive IP addresses from a VPC subnet secondary range, enabling direct VPC routing to pods.

Workload Identity — The recommended GKE mechanism for granting pods access to GCP APIs. It maps a Kubernetes service account to a GCP IAM service account, eliminating the need for service account key files.

YAML — A human-readable data serialization format used to write Kubernetes manifests (Deployments, Services, ConfigMaps, etc.) applied with `kubectl apply -f` .